

Full Stack Development

Name: Saurav Khadka | Student ID: 250784206

1. What Is Full Stack Development?

Full stack development refers to the development of both the front-end (client-side) and back-end (server-side) components of a web application. A full stack developer works across all layers of an application, including the user interface, server-side logic, and database systems, allowing them to build complete web applications from start to finish. It implies a full stack developer can design engaging and usable user interfaces, build secure and scalable systems to process the data and return responses from the server. Full stack developers are in demand in today's tech industry as they can handle the entire development process. From gathering a project's requirements and planning its architecture to putting the application live maintaining applications over time.

The role of a full stack developer will usually cover all areas of these activities. In a typical full stack application, there are several layers: the front end is for the end user to interact with, the back end is for implementing business logic and managing information, and the database layer is for data storage and retrieval. These layers will enable the ability to create a complete end-to-end solution that functions seamlessly as a whole.

1.1 Front End (Client-Side)

The front end refers to everything that a user sees and interacts with directly in their web browser. It ensures that websites and applications display neatly on the user's computer. People who develop front ends are responsible for the structure, visual design and interactive behaviour of a website or app.

Ensuring accessibility for all users, including those with disabilities; laying out responsive designs that work well on mobile, tablet and desktop computers; optimizing performance by minimizing assets and loading resources lazily also including appropriate Client-side validation giving users an easy and safe to use platform.

Core Technologies:

- HTML (HyperText Markup Language) is used to build a structure of a web page.
- CSS (Cascading Style Sheets) is used to control layout, colours and typography to produce a consistent look throughout the website or application.
- JavaScript is used to make a webpage interactive.
- Front-end development also frequently uses frameworks and libraries such as Bootstrap for responsive layouts.
- Nowadays, Angular, Vue or react are used to build single-page applications with complex features.

1.2 Back End (Server-Side)

The back end is the part of an application that is hidden from view of end users and run on a server. It houses business logic to process user requests, perform database operations, apply regulations and then send responses back to the front end.

Authenticating and authorizing users, implementing complex workflows, integrating with third-party APIs, managing sessions, and enforcing security policies are some responsibilities of a backend developer. To make a server secure Sanitizing and validating all input to prevent injection attacks, encrypting data in transit with HTTPS/TSL, applying the principle of least privilege for resource access, using secure storage for environment variables and credentials, as well as monitoring any suspicious activity through logging the activities to a database should be implemented.

Core Technologies:

- Common back-end environments include Node.js w/ Express, Python with Django and Flask, PHP using Laravel.
- REST APIs and GraphQL are used for communication between the front end and back end.
- To Enable real-time communication for applications such as chat systems and live notifications WebSockets is used.

1.3 Database & Data Layer

The database layer is responsible for storing, organising, retrieving, and updating data used by an application. It acts as the central storage system for information such as user accounts, products, orders, and application content. Users do not interact with the database directly; instead, all database operations are handled through the back end to maintain security and data integrity.

Database Types:

Relational (SQL) Databases

- MySQL/MariaDB
- PostgreSQL
- Microsoft SQL Server

These databases store information in structured tables with predefined relationships. They are well suited for applications that require reliable transactions and consistent data management.

Non-Relational (NoSQL) Databases

- MongoDB
- DynamoDB

These databases store data in flexible formats such as documents or key-value pairs. They are often used for applications that need to scale quickly and handle large volumes of data.

Back-end applications communicate with databases using queries and specialised data access tools to store, retrieve, and manage information efficiently. Technologies such as Sequelize, Prisma, and Mongoose simplify database interactions by providing developers with easier ways to work with data without writing complex queries manually. To improve performance, databases use techniques such as indexing and query optimisation, which help retrieve information more quickly. In addition, database migrations are used to safely manage changes to the database structure over time, ensuring that updates can be applied consistently without affecting existing data.

Database security involves restricting access through user permissions, encrypting sensitive data during storage and transmission, validating user input, performing regular backups to prevent data loss, and maintaining logs to monitor activity and detect potential security threats.

1.4 How the Layers Work Together

The front end, back end, and database are all integrated into one system called a full stack application. Any action a user takes on the front end is relayed to the back end, for example, when a user submits a form. The request is processed in the back end where the data is validated and if required, communicated to the database. This will return the data requested by the database to the back end, and the back end will send back a response to the front end. This interaction allows users to work with dynamic applications while keeping data secure and organised.

2. Full Stack Demonstration – NoteSpace (Note Keeper Application)

Live Demo: <https://saurav.freepage.cc/index.php>

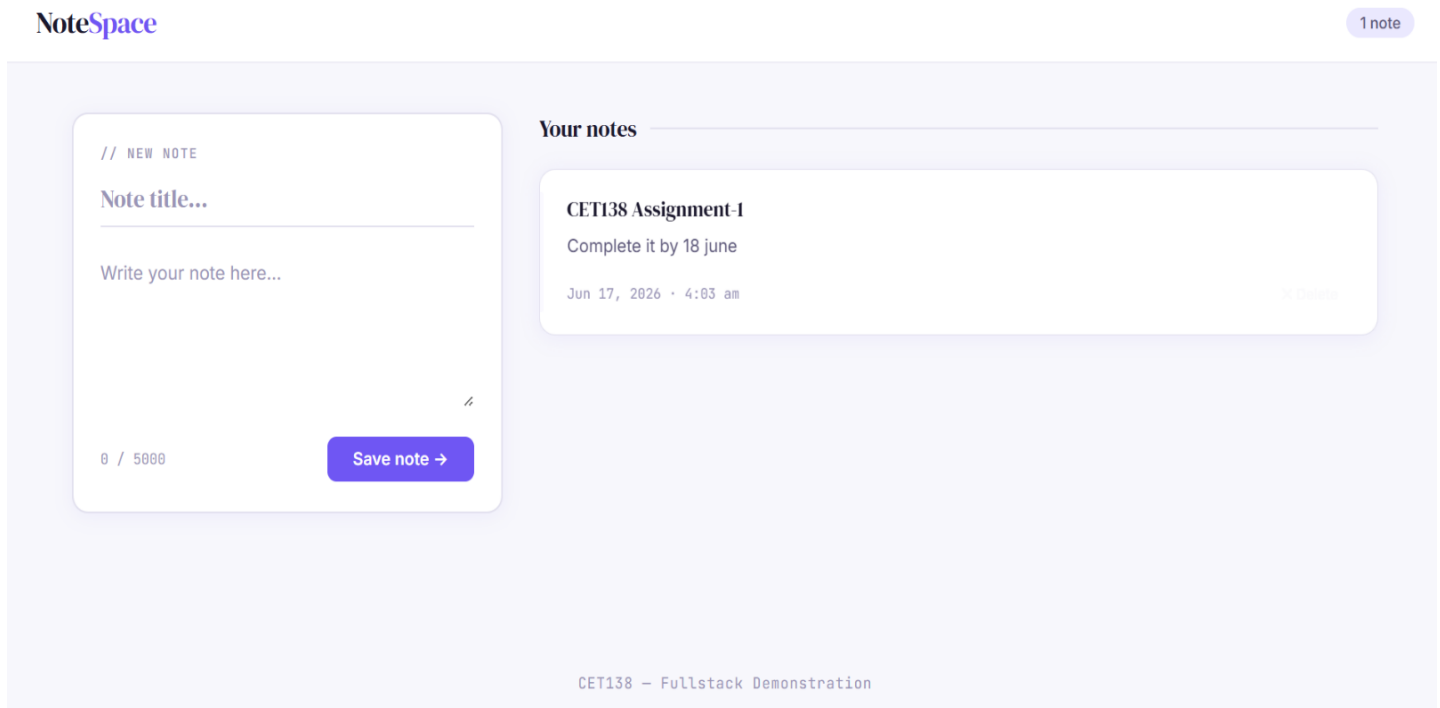


Figure 1: Full Stack Demonstration– Note Keeper Application

For my Full Stack Development demonstration, I created a Note Keeper application that allows users to create, view, and delete notes. The project was developed using HTML, CSS, and JavaScript for the user interface, PHP for server-side processing, and MySQL for data storage. The application demonstrates how front-end, back-end, and database technologies work together to create a complete web application.

2.1 How the Application Works and What It Demonstrates

The application allows users to enter a note title and content through a web form. When the user submits the form, the data is sent to the PHP back end for processing. The server validates the submitted information before storing it in a MySQL database.

When the homepage loads, PHP retrieves all stored notes from the database and displays them on the webpage. Users can also delete notes, which sends a request to the server and removes the selected record from the database.

This project demonstrates the complete flow of a full stack application. The front end collects user input, the backend processes requests and applies validation, and the database stores and retrieves information. All three layers work together to provide a functional and interactive user experience.

2.2 Technologies Used

Front End

- HTML was used to structure the application interface.
- CSS was used to control the appearance, layout, and responsiveness of the application.
- JavaScript was used to improve interactivity through features such as the live character counter, delete confirmation dialog, automatic message dismissal, and user interface updates.

Back End

- PHP was used to process form submissions and communicate with the database.
- Form validation was implemented to ensure users provide valid information before data is stored.
- PHP redirects were used to manage application flow after actions such as adding or deleting notes.

Database Layer

- MySQL was used to store note information permanently.
- SQL queries were used to insert, retrieve, and delete records.
- Prepared statements were used to improve security and protect against SQL injection attacks.

2.3 Features Implemented

- Create new notes
- View all saved notes.
- Delete existing notes.
- Input validation for note creation.
- Database storage and retrieval.
- Responsive user interface.
- Secure database operations using prepared statements.
- Live character counter
- Delete confirmation dialog
- Automatic success/error notifications

2.4 Reflection

Through this project, I gained practical experience developing a complete full stack application. I learned how front-end technologies, server-side scripting, and databases work together to process user requests and manage data. Developing the Note Keeper application improved my understanding of form handling, database interactions, input validation, and secure data management. This project helped me understand the responsibilities of a full stack developer and how multiple technologies can be combined to create functional web applications.